

BANCO DE DADOS

com Microsoft SQL Server

AULA 03

Teoria dos Conjuntos Aplicada ao SQL

A matemática por trás do modelo relacional

Pertinência

Inclusão

União

Interseção

Diferença

Complemento



Objetivos da Aula 03

1 2
3 4

Compreender a Teoria dos Conjuntos como base do modelo relacional



Entender pertinência e aplicá-la com IN e EXISTS no SQL



Identificar subconjuntos usando filtros e subconsultas



Aplicar a operação de União com UNION e UNION ALL



Aplicar Interseção com INTERSECT



Aplicar Diferença com EXCEPT



Trabalhar com Complemento usando NOT IN e NOT EXISTS



Relacionar cada operação matemática com casos reais de negócio



Revisão — Aula 02: DDL

? O que faz o ALTER TABLE ADD?

✓ Adiciona coluna ou constraint a uma tabela existente

? Diferença entre DROP e TRUNCATE?

✓ DROP remove estrutura; TRUNCATE remove dados (DDL, sem rollback, IDENTITY reseta)

? O que é uma CHECK constraint?

✓ Valida expressão lógica — ex: preço > 0, status IN lista de valores

? ON DELETE CASCADE faz o quê?

✓ Exclui automaticamente os filhos quando o registro pai é excluído

? Como nomear constraints corretamente?

✓ Padrão: PK_TABELA, FK_TAB1_TAB2, UQ_TAB_COL, CK_TAB_COL, DF_TAB_COL

O que é um Conjunto? — A Base Matemática do SQL



Definição: Um conjunto é uma coleção bem-definida de elementos distintos. Em SQL, cada tabela é um conjunto de linhas (tuplas) e cada SELECT retorna um novo conjunto de resultados.

Tabela CLIENTES

id	nome	estado
1	Ana Silva	SP
2	Bruno Santos	RJ
3	Carla Oliveira	MG
4	Diego Ferreira	PR

```
SELECT *  
FROM CLIENTE  
WHERE estado
```



Resultado = Subconjunto

id	nome	estado
1	Ana Silva	SP
6	Felipe Lima	SP

Conceitos fundamentais:

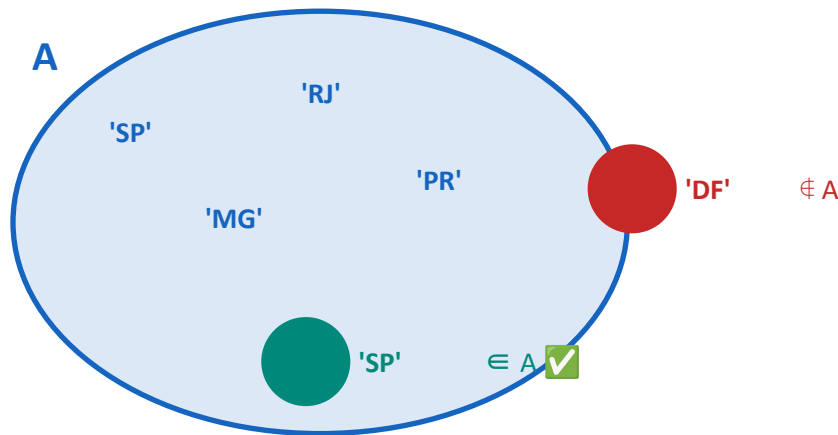
► Tabela = Conjunto: Cada linha é um elemento único do conjunto

► SELECT = Operação: Retorna um novo conjunto a partir de outros

► WHERE = Filtro: Define quais elementos pertencem ao resultado

∈ Pertinência — 'x pertence ao conjunto A?'

Pertinência ($\in$): dado um conjunto A, perguntamos se um elemento x pertence a A. Em SQL: o valor está na lista ou no resultado de outra query?



SQL — Pertinência

```
-- ∈ com lista (IN)
SELECT nome FROM CLIENTES
WHERE estado IN ('SP', 'RJ', 'MG');

-- ∈ com subquery
SELECT nome FROM CLIENTES
WHERE id_cliente IN (
    SELECT id_cliente FROM PEDIDOS);

-- ∉ Não-pertinência (NOT IN)
SELECT nome FROM CLIENTES
WHERE estado NOT IN ('SP', 'RJ');
```



CUIDADO: NOT IN com NULL na subquery retorna ZERO linhas! Sempre adicione WHERE coluna IS NOT NULL na subquery.



EXISTS vs IN — Qual usar?

```
-- IN: testa se o valor está na lista retornada
-- EXISTS: testa se a subquery retorna ALGUM resultado

-- IN: Clientes que fizeram pedido
SELECT nome FROM CLIENTES
WHERE id_cliente IN (SELECT id_cliente FROM PEDIDOS);

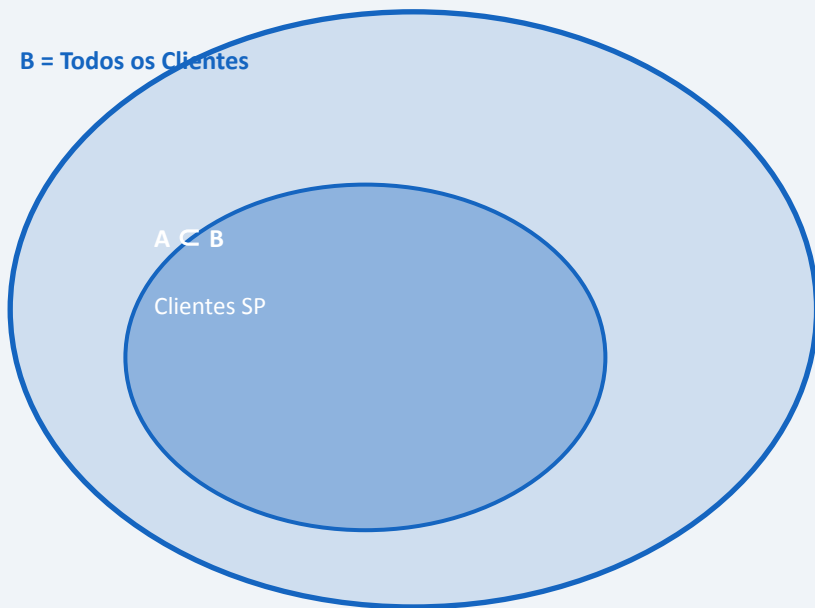
-- EXISTS: equivalente, mas para quando existe ao menos 1 registro
SELECT nome FROM CLIENTES c
WHERE EXISTS (SELECT 1 FROM PEDIDOS p
              WHERE p.id_cliente = c.id_cliente);

-- NOT EXISTS: clientes que NUNCA fizeram pedido
SELECT nome FROM CLIENTES c
WHERE NOT EXISTS (SELECT 1 FROM PEDIDOS p
                  WHERE p.id_cliente = c.id_cliente);

-- Diferença de performance:
-- IN → avalia TODOS os resultados da subquery antes de comparar
-- EXISTS → para no PRIMEIRO resultado encontrado (mais rápido!)
-- Regra prática:
-- Use IN quando a lista é pequena/fixa
-- Use EXISTS quando a subquery pode retornar muitos registros
-- Use NOT EXISTS (nunca NOT IN quando pode ter NULL na subquery!)
```

⊂ Inclusão — 'A está contido em B?'

Inclusão ($\subset$): dizemos que $A \subset B$ quando todo elemento de A também está em B. Em SQL: o resultado de uma query está completamente contido no resultado de outra?



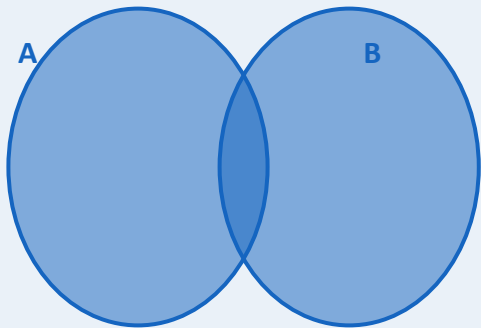
SQL — Verificar Inclusão

```
-- A  $\subset$  B: todo elemento de A está em B?  
-- Ex: todos os clientes SP fizeram pedido?  
  
-- Quem está em A mas não em B (diferença)?  
-- Se vazio  $\rightarrow$  A  $\subset$  B (inclusão confirmada)  
SELECT nome FROM CLIENTES  
WHERE estado = 'SP'  
AND id_cliente NOT IN (  
    SELECT id_cliente FROM PEDIDOS  
    WHERE id_cliente IS NOT NULL);  
  
-- Se retornar vazio: TODOS os clientes SP  
-- fizeram pedido  $\rightarrow$  A  $\subset$  B ✓  
-- Se retornar linhas: existem exceções  $\rightarrow$  A  $\not\subset$  B ✗
```

Casos práticos de Inclusão no dia a dia:

Diagrama de Venn — As 6 Operações de Conjunto em SQL

$\cup$ União



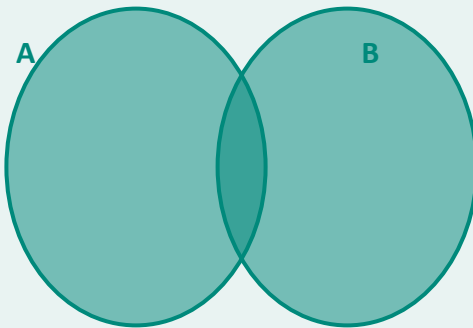
UNION / UNION ALL

Todos de A e B

 **Exemplo:**

Clientes + Funcionários em lista única

$\cap$ Interseção



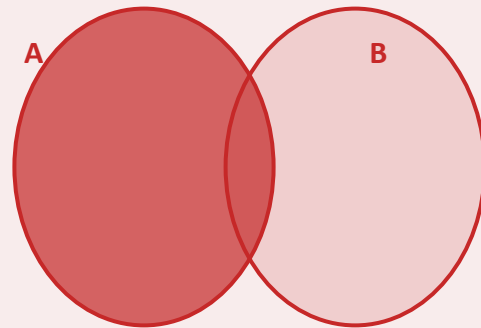
INTERSECT

Comuns a A e B

 **Exemplo:**

Clientes que também são fornecedores

$\setminus$ Diferença



EXCEPT

Em A mas não em B

 **Exemplo:**

Produtos nunca vendidos (catálogo - vendas)

U União — UNION e UNION ALL

```
-- UNIÃO (U): todos os elementos de A E de B
-- UNION      → elimina duplicatas (mais lento: ordena e compara)
-- UNION ALL → mantém tudo (mais rápido: sem verificação de dup.)

-- Regras obrigatórias:
-- 1. Mesmo número de colunas em cada SELECT
-- 2. Tipos de dados compatíveis (mesma posição)
-- 3. Nomes das colunas = nomes do 1º SELECT

-- Ex 1: Lista unificada de pessoas (UNION — sem duplicatas)
SELECT nome, 'Cliente'      AS tipo, cidade FROM CLIENTES
UNION
SELECT nome, 'Funcionário' AS tipo, NULL   FROM FUNCIONARIOS
ORDER BY tipo, nome; -- ORDER BY só no final!

-- Ex 2: Relatório de valores (UNION ALL — mantém duplicatas)
SELECT 'Produto'      AS origem, nome_produto AS descricao, preco AS valor
FROM PRODUTOS
UNION ALL
SELECT 'Funcionário' AS origem, nome          AS descricao, salario AS valor
FROM FUNCIONARIOS
ORDER BY valor DESC;

-- Quando usar UNION ALL?
-- Fontes DIFERENTES (produtos ≠ salários → nunca duplicam)
-- Ou quando você JÁ SABE que não há duplicatas

-- → UNION ALL é SEMPRE mais rápido que UNION
```

∩ Interseção — INTERSECT

```
-- INTERSEÇÃO (∩): elementos presentes em A E em B
-- INTERSECT elimina duplicatas automaticamente
-- Retorna apenas linhas idênticas nos dois SELECT

-- Ex 1: Pessoas que são CLIENTES e FUNCIONÁRIOS (mesmo nome)
SELECT nome FROM CLIENTES
INTERSECT
SELECT nome FROM FUNCIONARIOS;

-- Ex 2: Categorias que têm produtos E tiveram vendas
SELECT id_categoria FROM PRODUTOS
INTERSECT
SELECT pr.id_categoria
FROM   ITENS_PEDIDO ip
JOIN   PRODUTOS pr ON ip.id_produto = pr.id_produto;

-- Equivalente com JOIN (alternativa sem INTERSECT):
SELECT DISTINCT c.nome FROM CLIENTES c
JOIN FUNCIONARIOS f ON c.nome = f.nome;

-- Equivalente com IN (outra alternativa):
SELECT nome FROM CLIENTES
WHERE  nome IN (SELECT nome FROM FUNCIONARIOS);

-- INTERSECT → legível para consultas de cruzamento
-- JOIN → mais flexível, permite colunas adicionais
```

\ Diferença — EXCEPT

```
-- DIFERENÇA (\): elementos de A que NÃO estão em B
-- A EXCEPT B ≠ B EXCEPT A  (não é comutativa!)
-- EXCEPT elimina duplicatas automaticamente

-- Ex 1: Clientes que NUNCA fizeram nenhum pedido
SELECT id_cliente, nome FROM CLIENTES
EXCEPT
SELECT DISTINCT c.id_cliente, c.nome
FROM   CLIENTES c
JOIN   PEDIDOS  p ON c.id_cliente = p.id_cliente;

-- Ex 2: Produtos que NUNCA foram vendidos (sem saída)
SELECT id_produto, nome_produto FROM PRODUTOS
EXCEPT
SELECT DISTINCT pr.id_produto, pr.nome_produto
FROM   PRODUTOS      pr
JOIN   ITENS_PEDIDO ip ON pr.id_produto = ip.id_produto;

-- Ex 3: Categorias SEM produtos cadastrados
SELECT id_categoria, nome_categoria FROM CATEGORIAS
EXCEPT
SELECT DISTINCT c.id_categoria, c.nome_categoria
FROM   CATEGORIAS c
JOIN   PRODUTOS  p ON c.id_categoria = p.id_categoria;
```

Complemento — NOT IN e NOT EXISTS

Complemento ($\bar{A}$): conjunto de todos os elementos do universo que NÃO pertencem a A. Em SQL: registros que não satisfazem uma condição ou não aparecem em outra tabela/subquery.

NOT IN

```
SELECT nome FROM CLIENTES
WHERE id_cliente NOT IN (
    SELECT id_cliente
    FROM PEDIDOS
    WHERE id_cliente IS NOT NULL -- ⚠️
);
```

✅ Legível para listas pequenas

⚠️ Perigoso com NULL na subquery

NOT EXISTS

```
SELECT nome FROM CLIENTES c
WHERE NOT EXISTS (
    SELECT 1 FROM PEDIDOS p
    WHERE p.id_cliente =
        c.id_cliente
);
```

✅ Mais seguro (NULL não afeta)

⚠️ Sintaxe um pouco mais longa

Tabela Comparativa — Todas as Operações de Conjunto

Operação	Símbolo	SQL Server	Elimina dup.?	Retorna	Exemplo de uso
União	$\cup$	UNION	✓ Sim	Tudo de A e B	Lista unificada clientes+funcionários
União total	$\cup$	UNION ALL	✗ Não	Tudo de A e B	Relatório consolidado com duplicatas
Interseção	$\cap$	INTERSECT	✓ Sim	Apenas comuns	Quem é cliente E fornecedor
Diferença	$\setminus$	EXCEPT	✓ Sim	A menos B	Produtos nunca vendidos
Pertinência	$\in$	IN / EXISTS	—	Filtra linhas	Cientes que fizeram pedido
Não-pertinência	$\notin$	NOT IN / NOT EXISTS	—	Filtra linhas	Cientes SEM pedido



Casos Reais — Combinando Operações no SISTEMA_VENDAS

```
-- CASO 1: Relatório de 'saúde' - clientes inativos (sem pedido)
SELECT id_cliente, nome, cidade, estado FROM CLIENTES
EXCEPT
SELECT DISTINCT c.id_cliente, c.nome, c.cidade, c.estado
FROM   CLIENTES c JOIN PEDIDOS p ON c.id_cliente = p.id_cliente;

-- CASO 2: Lista completa de pessoas cadastradas (nome + função)
SELECT nome, email,      'Cliente'      AS perfil FROM CLIENTES
UNION
SELECT nome, NULL,       'Funcionário' AS perfil FROM FUNCIONARIOS
ORDER BY perfil, nome;

-- CASO 3: Produtos vendidos em Jan E em Fev (compradores fiéis)
SELECT DISTINCT ip.id_produto FROM ITENS_PEDIDO ip
JOIN   PEDIDOS p ON ip.id_pedido=p.id_pedido
WHERE  MONTH(p.data_pedido)=1
INTERSECT
SELECT DISTINCT ip.id_produto FROM ITENS_PEDIDO ip
JOIN   PEDIDOS p ON ip.id_pedido=p.id_pedido
WHERE  MONTH(p.data_pedido)=2;

-- CASO 4: Funcionários sem nenhum pedido registrado
SELECT nome FROM FUNCIONARIOS
WHERE NOT EXISTS (
    SELECT 1 FROM PEDIDOS WHERE id_funcionario=FUNCIONARIOS.id_funcionario);
```



Exercícios Propostos — Aula 03

01

Básico

Use IN para listar todos os clientes de SP, RJ ou MG. Depois reescreva usando OR para comparar os dois resultados e confirmar que são idênticos.

02

Básico

Use UNION para criar uma lista unificada com nome, 'Cliente' ou 'Funcionario' como tipo, e cidade de todas as pessoas cadastradas. Ordene por tipo e nome.

03

Médio

Use EXCEPT para encontrar todos os clientes que NUNCA fizeram um pedido. Depois resolva o mesmo problema com NOT EXISTS e compare as duas queries.

04

Médio

Use INTERSECT para descobrir quais categorias têm ao menos um produto cadastrado E ao menos uma venda registrada em ITENS_PEDIDO.

05

Avançado

Crie um relatório com UNION ALL mostrando: id, nome, 'Cliente', data_cadastro de CLIENTES e id, nome, 'Funcionario', data_admissao de FUNCIONARIOS, ordenado pela data mais recente. Use alias para padronizar as colunas.



Respostas Comentadas — Exercícios 02 e 03

```
-- EX 02: Lista unificada com UNION
SELECT nome, cidade, 'Cliente'      AS tipo FROM CLIENTES
UNION
SELECT nome, NULL,   'Funcionário' AS tipo FROM FUNCIONARIOS
ORDER BY tipo, nome;

-- Resultado: lista única sem duplicatas, ordenada

-- EX 03A: Clientes sem pedido — EXCEPT
SELECT id_cliente, nome FROM CLIENTES
EXCEPT
SELECT DISTINCT c.id_cliente, c.nome
FROM   CLIENTES c JOIN PEDIDOS p ON c.id_cliente = p.id_cliente;

-- EX 03B: Mesmo resultado — NOT EXISTS (mais seguro com NULL)
SELECT id_cliente, nome FROM CLIENTES c
WHERE NOT EXISTS (
    SELECT 1 FROM PEDIDOS p WHERE p.id_cliente = c.id_cliente);

-- Comparação: os dois retornam o mesmo resultado!
-- EXCEPT → mais simples de escrever e ler
-- NOT EXISTS → mais performático e seguro com NULLs
-- Escolha: use NOT EXISTS em produção e EXCEPT em análises rápidas
```



Desafio da Aula 03

 Contexto: Você foi chamado para fazer um relatório de 'Diagnóstico do SISTEMA_VENDAS' usando exclusivamente operações de conjuntos. O gerente quer saber 'o que está faltando e o que está sobrando' no sistema.

Consultas obrigatórias:

EXCEPT

Clientes sem nenhum pedido (potenciais para campanha de reativação)

EXCEPT

Produtos que nunca foram vendidos (revisar catálogo ou criar promoção)

EXCEPT

Categorias sem nenhum produto cadastrado (categoria vazia)

INTERSEÇÃO

Funcionários que também são clientes (pelo nome — cross-reference)

UNION ALL

Relatório consolidado: produtos (nome, preço) + funcionários (nome, salário) com coluna 'tipo'

NOT EXISTS

Pedidos que não têm nenhum item em ITENS_PEDIDO (pedidos 'fantasmas')

BÔNUS

Combine EXCEPT + UNION: clientes sem pedido de SP e RJ em uma única lista



Resumo — Aula 03: Teoria dos Conjuntos no SQL



Pertinência

IN (lista/subquery) — verifica se valor pertence ao conjunto



União — UNION

Todos de A e B. UNION ALL é mais rápido (mantém duplicatas)



Interseção — INTERSECT

Apenas elementos presentes em A E em B. Elimina duplicatas



Diferença — EXCEPT

Elementos de A que NÃO estão em B. $A \neq B$ (não comutativo)



Complemento

NOT IN / NOT EXISTS — o que não pertence ao conjunto



Armadilha do NULL

NOT IN com NULL na subquery retorna ZERO linhas! Use NOT EXISTS



Performance

EXISTS > IN (grandes volumes). UNION ALL > UNION. NOT EXISTS > NOT IN



Regras UNION/INTERSECT/EXCEPT

Mesmo nº colunas, tipos compatíveis, ORDER BY só no final



Próxima Aula 04: DML Parte I — INSERT

Aula 04 — O que vem por aí:



INSERT básico

INSERT INTO tabela (colunas) VALUES (valores) — com boas práticas



INSERT múltiplo

Inserir N registros em um único comando — mais eficiente



INSERT...SELECT

Copiar dados de uma tabela para outra — migração e backup



IDENTITY e SCOPE_IDENTITY()

Como capturar o ID gerado automaticamente



OUTPUT

Ver os valores inseridos sem precisar de outra query



Popular SISTEMA_VENDAS

Inserir dados reais em todas as 6 tabelas do projeto



Para a próxima aula: conclua o desafio da Aula 03 e revise os dados do SISTEMA_VENDAS — vamos populá-lo com dados reais!